

SIMATS ENGINEERING

ASSESSMENT TOOL 3

ARTIFICIAL INTELLIGENCE

Course Code: CSA17

STUDENT NAME

Shanmuganathan S R S

REGISTER NUMBER

192511347

CO4 – Integrated Experiments

Banking – Loan Default Prediction

Student: Shanmuganathan S R S **Register No:** 192511347

The assigned scenario is Banking – Loan Default Prediction. The assessment asks for a Decision Tree and Multi-Layer Perceptron classifier, followed by a Q-Learning agent for account-management actions.

filecite turn0file0 L538-L553

Question 1 – Decision Tree vs Neural Network Classifier

Use income, credit score, loan amount and repayment history as inputs. Compare Accuracy, Precision, Recall and F1-Score, and plot the first three Decision Tree levels and the neural-network architecture.

filecite turn0file0 L540-L548

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt

df = pd.read_csv("loan_data.csv")
X = df[["income", "credit_score", "loan_amount", "repayment_history"]]
y = df["default"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

dt = DecisionTreeClassifier(max_depth=3, random_state=42)
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

mlp = MLPClassifier(hidden_layer_sizes=(16, 8), max_iter=1000, random_state=42)
mlp.fit(X_train_s, y_train)
mlp_pred = mlp.predict(X_test_s)

def scores(y, p):
    return (accuracy_score(y, p), precision_score(y, p, zero_division=0),
            recall_score(y, p, zero_division=0), f1_score(y, p, zero_division=0))

print("Decision Tree:", scores(y_test, dt_pred))
print("MLP:", scores(y_test, mlp_pred))

plt.figure(figsize=(14, 8))
plot_tree(dt, feature_names=X.columns,
          class_names=["No Default", "Default"], max_depth=3, filled=True)
plt.title("Decision Tree - First 3 Levels")
plt.show()

print("MLP architecture: 4 inputs -> 16 -> 8 -> output")
```

Results table: Run the above code on the supplied loan dataset and enter the generated values.

Model	Accuracy	Precision	Recall	F1-Score
Decision Tree	Run code	Run code	Run code	Run code
MLP	Run code	Run code	Run code	Run code

Justification: The Decision Tree is suitable for real-time loan screening when interpretability, fast inference and easy business-rule explanation are priorities. The MLP can learn more complex nonlinear relationships, but requires feature scaling and is less transparent. Therefore, the Decision Tree is the practical choice when explainability is important; otherwise, select the model that demonstrates better validated performance on the actual dataset.

Question 2 – Reinforcement Learning for Account Management

The Q-Learning state is the predicted default-risk level. The available actions are Approve, Reject, Restructure and Monitor. The trained Q-table and learned policy are then connected to the classifier output.

■filecite■turn0file0■L549-L553■

```
import numpy as np, random

actions=["Approve","Reject","Restructure","Monitor"]
Q=np.zeros((3,4)) # 0=Low, 1=Medium, 2=High
alpha,gamma,epsilon=0.1,0.9,0.2

def reward(state,action):
    R={
        0:{"Approve":5,"Monitor":2,"Restructure":0,"Reject":-3},
        1:{"Approve":-2,"Monitor":4,"Restructure":5,"Reject":0},
        2:{"Approve":-5,"Monitor":1,"Restructure":5,"Reject":6}}
    return R[state][action]

for episode in range(5000):
    state=random.randint(0,2)
    for step in range(10):
        a=random.randrange(4) if random.random()<epsilon else np.argmax(Q[state])
        r=reward(state,actions[a])
        next_state=random.randint(0,2)
        Q[state,a]+=alpha*(r+gamma*np.max(Q[next_state])-Q[state,a])
        state=next_state

print("Q-table:")
print(Q)

policy={state:actions[np.argmax(Q[state])] for state in range(3)}
print("Learned policy:",policy)

def risk_state(default_probability):
    if default_probability < 0.33: return 0
    if default_probability < 0.66: return 1
    return 2

# After dt.predict_proba(customer_features):
# p = dt.predict_proba(customer_features)[0,1]
# state = risk_state(p)
# action = actions[np.argmax(Q[state])]
# print(state, action)
```

Q-table: Rows represent Low, Medium and High default-risk states; columns represent Approve, Reject, Restructure and Monitor. The learned policy chooses the action with the maximum Q-value in each row.

End-to-end pipeline: Customer records → classifier → default probability → Low/Medium/High risk state → Q-table lookup → best account-management action.

Conclusion: The solution satisfies the required integrated design by combining supervised classification with reinforcement learning. Numerical metrics and the final Q-table must be generated from the actual loan dataset; the assessment document itself specifies the scenario and required outputs but does not provide those numerical results. ■filecite■turn0file0■L688-L757■